EIE330 FPGA-based System Design

Lab 1-1: Full Adder

Group 10

Member	Name	Student ID
1	[Name]	[Student ID]
2	[Name]	[Student ID]
3	[Name]	[Student ID]

Lab 1-1 | Introduction and Objectives

A full adder adds A, B and the carry-in Cin to produce the sum bit S and carry-out Cout. Unlike a half adder, it accepts a carry from the previous bit, so full adders can be connected to form a multi-bit adder[8].

This experiment uses three buttons for A, B and Cin, and two LEDs for S and Cout. All eight input combinations are tested[1]. The buttons and LEDs are active-low, so the design inverts the signals at both the input and output stages. Table 1-1.1 lists the board specifications[2]. The simulation and virtual-board test records show that all eight cases match the truth table.

Member Contributions

- Member 1: [Contribution to be added].
- Member 2: [Contribution to be added].
- Member 3: [Contribution to be added].

Design Constraints

Item	Specification
FPGA	Cyclone IV E EP4CE10F17C8N[2,4]
Resources	10,320 logic elements; 179 user I/O pins in the 256-pin package[3]
Buttons	Released: pin = 1; pressed: pin = 0; nominal 3.3 V pull-up[2,4]
LED	Pin = 0: LED on; pin = 1: LED off[1,2]
Clock	50 MHz board clock[2]; no clock or stored state is needed for this combinational circuit

Table 1-1.1. Board specifications and interface levels.

Lab 1-1 | Circuit Design

In the truth table, 1 means a button is pressed or an LED is on, although the corresponding pin is at logic 0. The suffix `_n` marks an active-low port. Table 1-1.2 lists the port connections, and Table 1-1.3 gives the truth table[1,2].

Port (1 bit)	I/O	Active level	Board signal / label
btn_a_n	Input	0 = A pressed	B2 / KEY1
btn_b_n	Input	0 = B pressed	B3 / KEY2
btn_cin_n	Input	0 = Cin pressed	B4 / KEY3
led_s_n	Output	0 = S LED on	led1 / LED0
led_cout_n	Output	0 = Cout LED on	led2 / LED1

Table 1-1.2. full_adder_lab port connections.

Logical inputs	Expected S, Cout
0 0 0	0 0
0 0 1	1 0
0 1 0	1 0
0 1 1	0 1
1 0 0	1 0
1 0 1	0 1
1 1 0	0 1
1 1 1	1 1

Table 1-1.3. Truth table (output 1 means LED on).

Figure 1-1.1 shows the circuit structure. The full-adder logic and the input and output inversions are all included in `full_adder_lab`. With only three inputs, two outputs and a small amount of logic, there is no need to split the circuit into submodules.

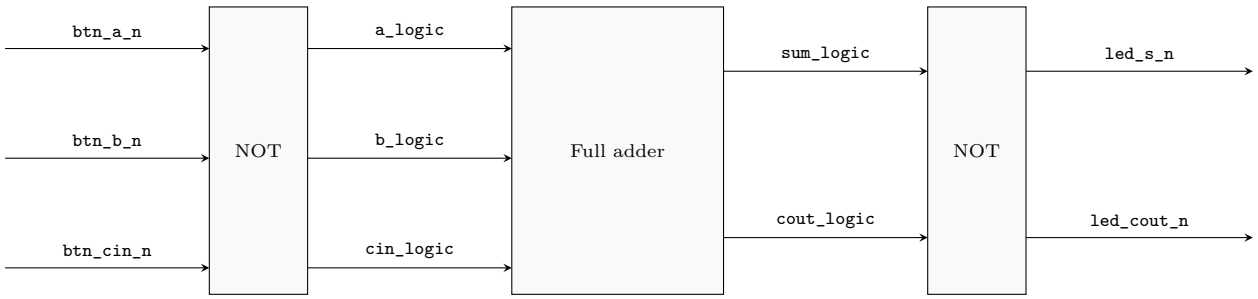


Figure 1-1.1. Input inversion, combinational logic and output inversion.

S is 1 when an odd number of inputs are 1, and Cout is 1 when at least two inputs are 1. The logic expressions are

$$S = A \oplus B \oplus C_{\text{in}}, \qquad C_{\text{out}} = AB + AC_{\text{in}} + BC_{\text{in}}.$$

Here, $\oplus$ means XOR, multiplication means AND, and $+$ means OR.

The outputs depend only on the current inputs, so no clock or registers are needed. The code uses a dataflow description, with `assign` statements that follow the logic expressions and can be checked against the truth table. The input pins are inverted to obtain A, B and Cin. The results are inverted again at the outputs so that the LEDs display the correct values.

Lab 1-1 | Implementation

Figure 1-1.2 shows the expected waveforms, with the inputs applied in truth-table order and each combination held for 10 ns.

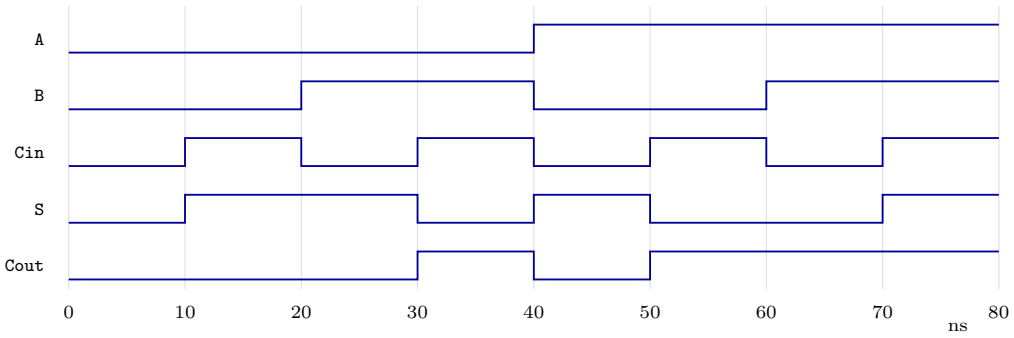


Figure 1-1.2. Expected input and output waveforms.

The design code is stored in RTL/ and the testbench in sim/. The following snippet shows the sum calculation and LED output. The complete code is available through the GitHub link below.

```
assign sum_logic = a_logic ^ b_logic ^ cin_logic;
// Invert the result to drive the active-low LED.
assign led_s_n = ~sum_logic;
```

RTL and testbench: [**GitHub link to be added**]

Icarus Verilog 12.0 was used for syntax checking[5], and Yosys 0.33 for synthesis[6]. The run logs report no warnings or errors.

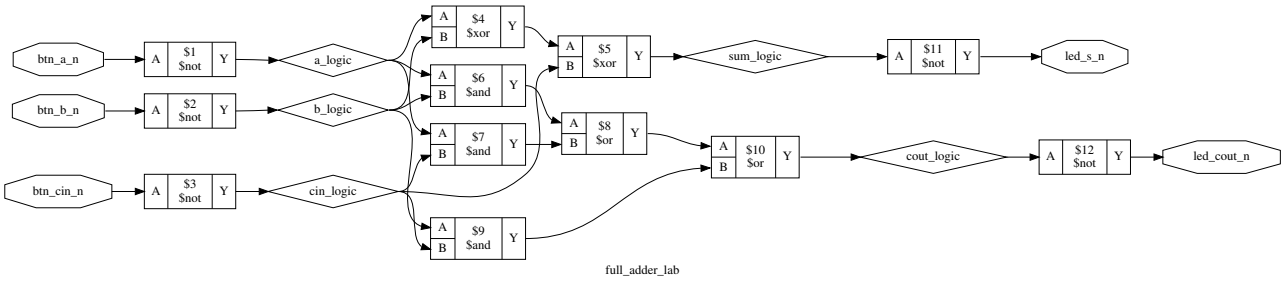


Figure 1-1.3. Yosys RTL view before generic gate mapping.

Yosys synthesis statistics	**Count**
Wires	10
Cells	7
Memories / processes	0 / 0

Table 1-1.4. Yosys synthesis statistics.

Figure 1-1.3 shows the input inversion, full-adder logic and output inversion, matching the design in Figure 1-1.1. Table 1-1.4 lists the synthesis results: 7 generic cells, with no latches or stored state. This is a generic Yosys cell count and does not directly represent the number of Cyclone IV logic elements used.

Lab 1-1 | Testing and Results

Testbench

The inputs are ordered as A, B, Cin and tested from 000 through 111. Since the full adder is a combinational circuit, these eight cases cover its complete truth table. The outputs are checked after each input change.

The testbench declares its input drivers as `reg` and its output signals as `wire`. An `initial` block changes the inputs every 10 ns. The input values are inverted before being applied to the button ports, and the LED outputs are inverted before comparison with the expected values. The `$dumpfile` and `$dumpvars` commands save the waveforms to a VCD file.

The expected values are calculated directly from A+B+Cin. The sum modulo 2 gives S, and Cout is 1 when the sum is at least 2. This provides a separate check of the XOR and carry expressions in the design code.

Table 1-1.5 compares the expected and simulated outputs for all eight cases. The LED pin levels have been converted back to logical values. All eight results match.

Logical inputs	Expected S, Cout	Actual S, Cout	Result
0 0 0	0 0	0 0	PASS
0 0 1	1 0	1 0	PASS
0 1 0	1 0	1 0	PASS
0 1 1	0 1	0 1	PASS
1 0 0	1 0	1 0	PASS
1 0 1	0 1	0 1	PASS
1 1 0	0 1	0 1	PASS
1 1 1	1 1	1 1	PASS

Table 1-1.5. Simulation results from the CSV file.

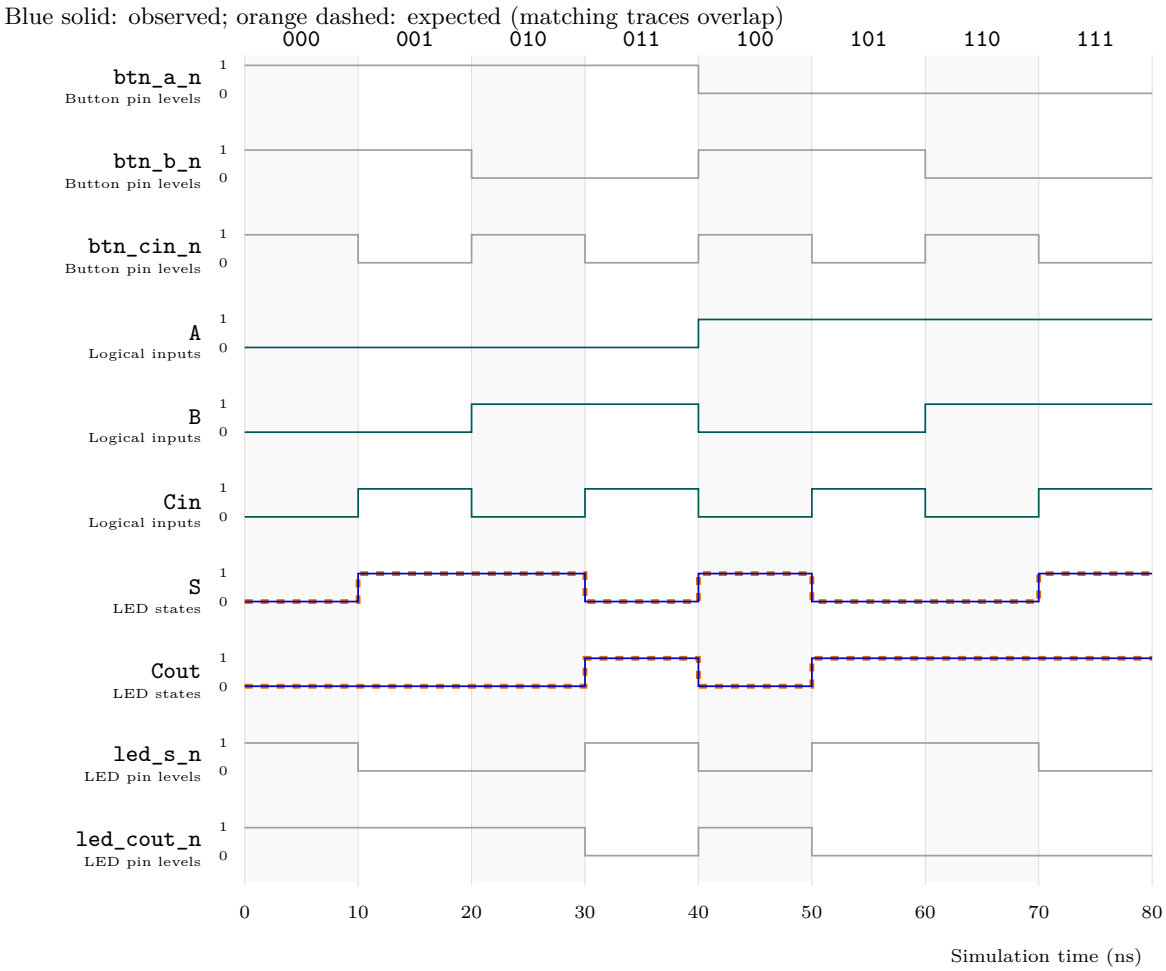
Waveform Inspection

GTKWave 3.3.116 was used to inspect both the logical signals and the pin levels. During the 40–50 ns interval, A=1, B=0 and Cin=0 produce S=1 and Cout=0, with LED pin levels of (0,1).

Figure 1-1.4 compares the waveforms plotted from the VCD data with the expected waveforms. S and Cout match the expected values in all eight input cases.

The 10 ns interval is a testbench setting, not the response time of the FPGA. This simulation does not measure the propagation delay of the physical circuit.

Lab 1-1 | Waveform Comparison



All 136 stable observations in the VCD record match the expected values.

Figure 1-1.4. Expected and VCD waveforms of the full adder.

A logical value of 1 means pressed or on; the corresponding pin level is 0.

Lab 1-1 | Virtual-board Testing and Summary

The virtual board runs the RTL through Verilator and uses SDL2 for its display. Its input handling was adjusted to allow several buttons to remain pressed at once: keyboard keys 2, 3 and 4 toggle B2, B3 and B4, respectively, and key 0 releases all buttons. These changes affect only the simulator input handling; the RTL was unchanged. The simulator is based on Simple-VGA-Simulator[7].

Table 1-1.6 lists the virtual-board results for all eight cases. Inputs are ordered as A, B, Cin. A value of 1 for S or Cout means the corresponding LED is on.

Logical inputs	Expected LED states	Result
000	S=0, Cout=0	PASS
001	S=1, Cout=0	PASS
010	S=1, Cout=0	PASS
011	S=0, Cout=1	PASS
100	S=1, Cout=0	PASS
101	S=0, Cout=1	PASS
110	S=0, Cout=1	PASS
111	S=1, Cout=1	PASS

Table 1-1.6. Full-adder virtual-board test results.

Virtual-board test video: **[Video link to be added]**

Local recording: full_adder/evidence/full_adder_verified-screen0.webm

The full-adder module does not use the remaining board signals. The top-level wrapper turns off the unused LEDs and sets the VGA outputs to an inactive state.

Discussion and Conclusion

All eight cases in the simulation and virtual-board tests match the full-adder truth table. For input 011, the two 1s add to binary 10, giving S=0 and Cout=1. For input 111, the result is binary 11, so both outputs are 1.

The distinction between logical values and pin levels matters in this experiment: a logical output of 1 requires an LED pin level of 0. Inverting the inputs and outputs makes the button and LED states agree with the truth table. Synthesis inferred no latches or stored state, as expected for a combinational design. Functional simulation and virtual-board testing were completed. Since functional simulation uses a zero-delay model, operation on a physical board and actual propagation delay still require hardware testing[1].

AI Use Statement

AI was used to help with language editing, formatting and information searches during the preparation of this report.

References

[1] Macau University of Science and Technology, "EIE330 FPGA-based System Design - Lab 1 Requirements," course handout, accessed Oct. 8, 2026.

[2] Macau University of Science and Technology, "EIE330 FPGA-based System Design: Lab 1-1 Simple LED Circuit," lecture slides, updated Oct. 5, 2026, accessed Oct. 8, 2026.

[3] Altera Corporation, "Cyclone IV FPGA Device Family Overview," Cyclone IV Device Handbook, vol. 1, doc. CYIV-51001-2.0, Mar. 2016, Tables 1-1, 1-3 and Fig. 1-3.

[4] Dongguan EmbedFire Electronic Technology Co., Ltd., "Wildfire EP4CE10 Pro schematic diagram," rev. V1.0, Sep. 14, 2023.

[5] Icarus Verilog contributors, "Getting Started With Icarus Verilog." [Online]. Available: https://steveicarus.github.io/iverilog/usage/getting_started.html. Accessed: Oct. 8, 2026.

[6] YosysHQ, "Synthesis starter." [Online]. Available: https://yosyshq.readthedocs.io/projects/yosys/en/latest/getting_started/example_synth.html. Accessed: Oct. 8, 2026.

[7] pikipity, "Simple-VGA-Simulator," GitHub repository. [Online]. Available: https://github.com/pikipity/Simple-VGA-Simulator. Accessed: Oct. 8, 2026. The local input modifications are included in the project source archive.

[8] Massachusetts Institute of Technology, "Lectures 8/9," 6.111 Introductory Digital Systems Laboratory, Spring 2006, slides 5, 8. [Online]. Available: https://web.mit.edu/6.111/www/s2006/LECTURES/l8_9.pdf. Accessed: Oct. 8, 2026.

EIE330 FPGA-based System Design

Lab 1-2: One-bit Digital Comparator

Group 10

Member	Name	Student ID
1	[Name]	[Student ID]
2	[Name]	[Student ID]
3	[Name]	[Student ID]

Lab 1-2 | Introduction and Objectives

A one-bit comparator indicates whether A is less than, equal to or greater than B through LT, EQ and GT. These outputs can be used as decision signals in a control circuit. In this experiment, LED1, LED2 and LED3 display the three results, and exactly one LED should be on for each input combination[1].

Two buttons provide A and B. The tests cover 00, 01, 10 and 11. Both the buttons and LEDs are active-low; Table 1-2.1 lists the board specifications[2]. The simulation and virtual-board records show the required LED states for all four inputs.

Member Contributions

- Member 1: [Contribution to be added].
- Member 2: [Contribution to be added].
- Member 3: [Contribution to be added].

Design Constraints

Item	Specification
FPGA	Cyclone IV E EP4CE10F17C8N[2,4]
Resources	10,320 logic elements; 179 user I/O pins in the 256-pin package[3]
Buttons	Released: pin = 1; pressed: pin = 0; nominal 3.3 V pull-up[2,4]
LED	Pin = 0: LED on; pin = 1: LED off[1,2]
Clock	50 MHz board clock[2]; no clock or stored state is needed for this combinational circuit

Table 1-2.1. Board specifications and interface levels.

Lab 1-2 | Circuit Design

A logical value of 1 again means a button is pressed or an LED is on, with a corresponding pin level of 0. Table 1-2.2 lists the port connections, and Table 1-2.3 gives the results for all four input combinations[1,2].

Port (1 bit)	I/O	Active level	Board signal / label
btn_a_n	Input	0 = A pressed	B2 / KEY1
btn_b_n	Input	0 = B pressed	B3 / KEY2
led_lt_n	Output	0 = LT LED on	led1 / LED0
led_eq_n	Output	0 = EQ LED on	led2 / LED1
led_gt_n	Output	0 = GT LED on	led3 / LED2

Table 1-2.2. comparator_lab port connections.

Logical inputs	Expected LT, EQ, GT
0 0	0 1 0
0 1	1 0 0
1 0	0 0 1
1 1	0 1 0

Table 1-2.3. Truth table (output 1 means LED on).

Figure 1-2.1 shows the comparator structure. The `comparator_lab` module includes the input inversion, comparison logic and output inversion. All three results can be expressed directly in terms of A and B, so a single functional module is sufficient.

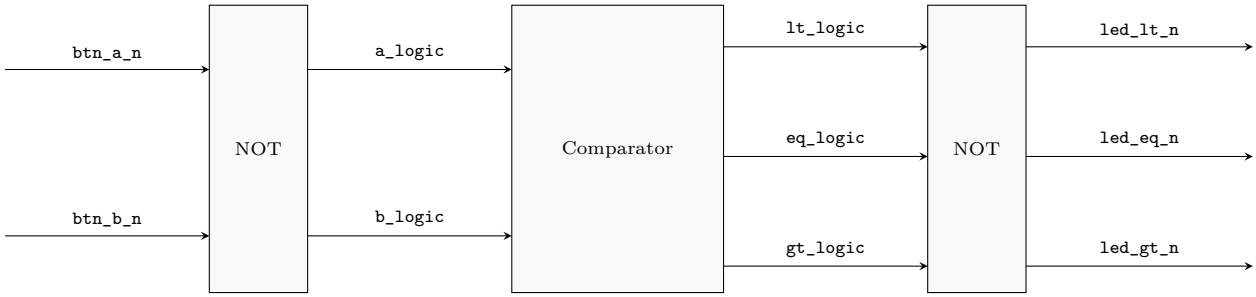


Figure 1-2.1. Input inversion, combinational logic and output inversion.

LT is 1 when A=0 and B=1; EQ is 1 when A and B are equal; and GT is 1 when A=1 and B=0. The outputs are

$$LT = \overline{A}B, \qquad EQ = \overline{A \oplus B}, \qquad GT = A\overline{B}.$$

Exactly one of these three conditions holds for each of the four input combinations.

The comparison depends only on the current values of A and B, so no clock or registers are needed. The code uses a dataflow description, with one `assign` expression for each result. The button inputs are inverted to obtain the logical values, and the results are inverted before being sent to the LED pins.

Lab 1-2 | Implementation

Figure 1-2.2 shows the expected waveforms for inputs 00, 01, 10 and 11, with each combination held for 10 ns.

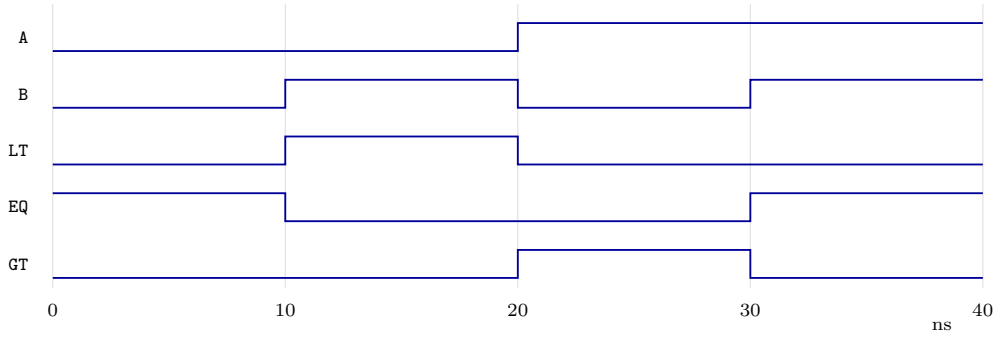


Figure 1-2.2. Expected input and output waveforms.

The comparator code is stored in RTL/ and the testbench in sim/. The snippet below shows the equality check and its LED output. The full code is available through the GitHub link.

```
assign eq_logic = ~(a_logic ^ b_logic);
assign led_eq_n = ~eq_logic;
```

RTL and testbench: **[GitHub link to be added]**

The comparator was checked with Icarus Verilog 12.0[5] and synthesized with Yosys 0.33[6]. Neither run log reports warnings or errors.

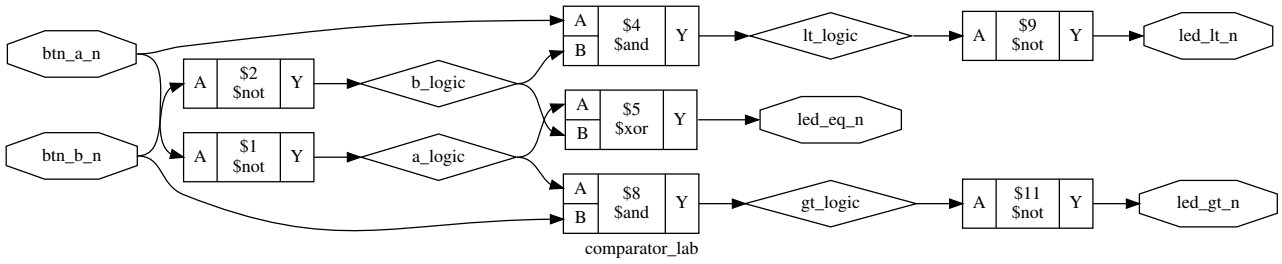


Figure 1-2.3. Yosys RTL view before generic gate mapping.

Yosys synthesis statistics	Count
Wires	5
Cells	3
Memories / processes	0 / 0

Table 1-2.4. Yosys synthesis statistics.

The ports and logic in Figure 1-2.3 correspond to Figure 1-2.1, although some inversions have been combined. For example, EQ is an inverted XOR result. Inverting it again for the LED output means that `led_eq_n` can be driven directly by the XOR result. Table 1-2.4 lists the final synthesis statistics: 3 generic cells, with no latches or stored state. This count does not represent the actual number of FPGA logic elements used.

Lab 1-2 | Testing and Results

Testbench

The inputs are ordered as A, B and tested as 00, 01, 10 and 11. The comparator has no internal state, so these four cases cover its complete truth table.

The testbench uses `reg` signals to drive the inputs and `wire` signals to observe the outputs. An `initial` block changes the inputs every 10 ns and inverts them before applying them to the button ports. The LED outputs are inverted before comparison with the expected values. The `$dumpfile` and `$dumpvars` commands save the waveforms to a VCD file.

The expected values are calculated by comparing A and B directly. The test checks LT, EQ and GT individually and also checks that exactly one output is 1.

Table 1-2.5 compares the expected and simulated values for all four inputs. The LED pin levels are inverted to obtain the logical outputs shown in the table. All four results match.

Logical inputs	Expected LT, EQ, GT	Actual LT, EQ, GT	Result
0 0	0 1 0	0 1 0	PASS
0 1	1 0 0	1 0 0	PASS
1 0	0 0 1	0 0 1	PASS
1 1	0 1 0	0 1 0	PASS

Table 1-2.5. Simulation results from the CSV file.

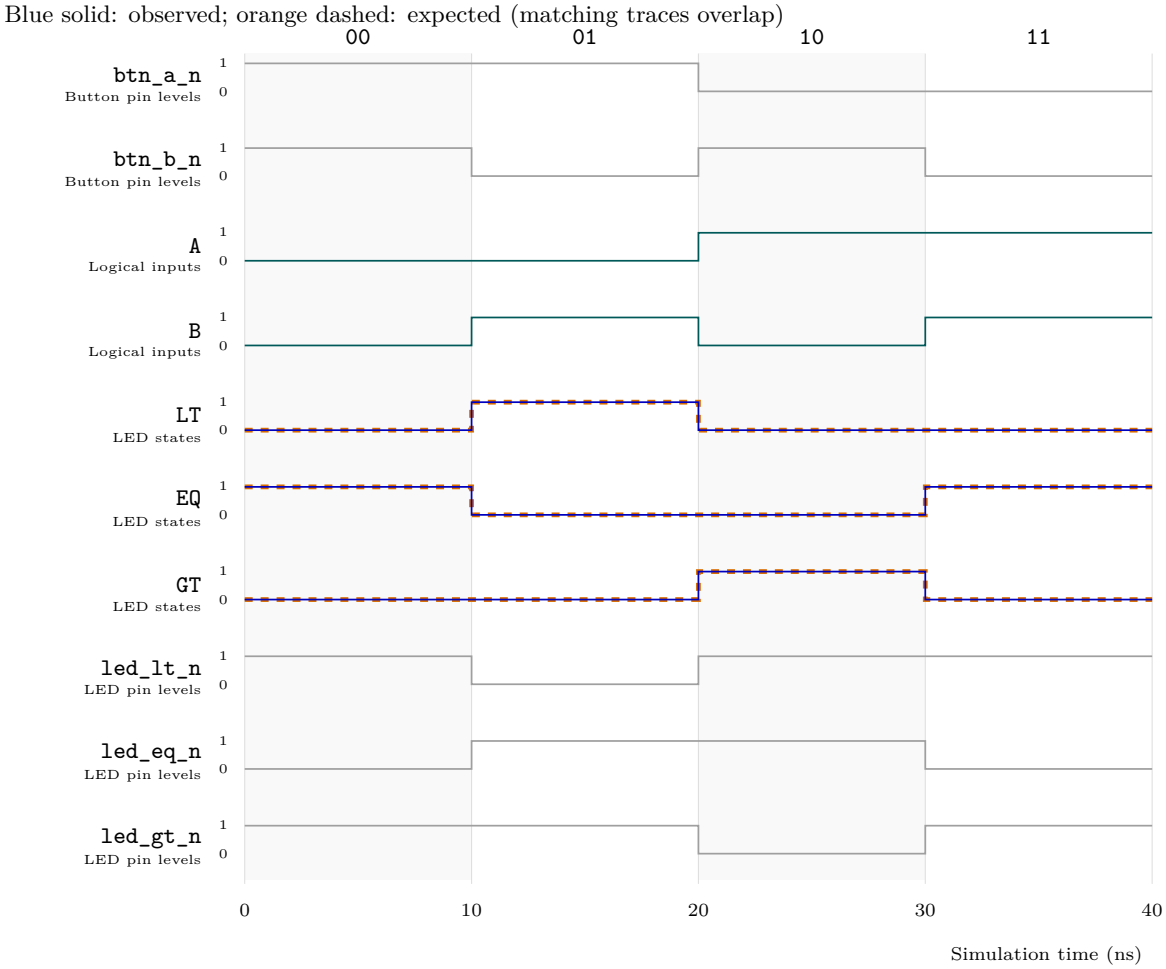
Waveform Inspection

In the GTKWave 3.3.116 waveform, A=0 and B=1 at 15.8 ns produce LT=1, EQ=0 and GT=0. The button pin levels are (1,0), and the LED pin levels are (0,1,1), so only the less-than LED is on.

Figure 1-2.4 overlays the waveforms plotted from the VCD data with the expected waveforms. LT, EQ and GT match the expected values in all four cases.

The 10 ns interval is again set by the testbench. These waveforms check the comparison results, but they cannot establish the propagation delay of the physical FPGA.

Lab 1-2 | Waveform Comparison



All 36 stable observations in the VCD record match the expected values.

Figure 1-2.4. Expected and VCD waveforms of the comparator.

A logical output of 1 corresponds to an LED pin level of 0. Both signals are shown for comparison.

Lab 1-2 | Virtual-board Testing and Summary

The comparator uses the same Verilator/SDL2 virtual-board environment. Keyboard keys 2 and 3 toggle B2 and B3, respectively, and key 0 releases all buttons. Both buttons can remain pressed to test input 11. The simulator is based on Simple-VGA-Simulator[7].

Table 1-2.6 lists the LED states for all four input combinations.

Logical inputs	Expected LED states	Result
00	Only LED2 on	PASS
01	Only LED1 on	PASS
10	Only LED3 on	PASS
11	Only LED2 on	PASS

Table 1-2.6. Comparator virtual-board test results.

Virtual-board test video: [**Video link to be added**]

Local recording: comparator/evidence/virtual_board-screen0.webm

The comparator module does not use the other board signals. The top-level wrapper turns off the remaining LEDs and sets the VGA outputs to an inactive state.

Discussion and Conclusion

All four results in the simulation and virtual-board test records match the truth table. Inputs 00 and 11 give EQ=1, input 01 gives LT=1, and input 10 gives GT=1. Exactly one LED is on in each case.

Equal inputs, 00 and 11, turn on only the EQ LED. Unequal inputs turn on either the LT or GT LED. Inverting the outputs makes each comparison result correspond to the correct LED state. These tests check the logical functions and the virtual-board display. Functional simulation uses a zero-delay model, so operation and propagation delay on a physical board still require hardware testing[1].

AI Use Statement

AI was used to help with language editing, formatting and information searches during the preparation of this report.

References

[1] Macau University of Science and Technology, "EIE330 FPGA-based System Design - Lab 1 Requirements," course handout, accessed Oct. 8, 2026.

[2] Macau University of Science and Technology, "EIE330 FPGA-based System Design: Lab 1-1 Simple LED Circuit," lecture slides, updated Oct. 5, 2026, accessed Oct. 8, 2026.

[3] Altera Corporation, "Cyclone IV FPGA Device Family Overview," Cyclone IV Device Handbook, vol. 1, doc. CYIV-51001-2.0, Mar. 2016, Tables 1-1, 1-3 and Fig. 1-3.

[4] Dongguan EmbedFire Electronic Technology Co., Ltd., "Wildfire EP4CE10 Pro schematic diagram," rev. V1.0, Sep. 14, 2023.

[5] Icarus Verilog contributors, "Getting Started With Icarus Verilog." [Online]. Available: https://steveicarus.github.io/iverilog/usage/getting_started.html. Accessed: Oct. 8, 2026.

[6] YosysHQ, "Synthesis starter." [Online]. Available: https://yosyshq.readthedocs.io/projects/yosys/en/latest/getting_started/example_synth.html. Accessed: Oct. 8, 2026.

[7] pikipity, "Simple-VGA-Simulator," GitHub repository. [Online]. Available: https://github.com/pikipity/Simple-VGA-Simulator. Accessed: Oct. 8, 2026. The local input modifications are included in the project source archive.

[8] Massachusetts Institute of Technology, "Lectures 8/9," 6.111 Introductory Digital Systems Laboratory, Spring 2006, slides 5, 8. [Online]. Available: https://web.mit.edu/6.111/www/s2006/LECTURES/l8_9.pdf. Accessed: Oct. 8, 2026.